INFORME DEL TERCER AVANCE

Sistema Web de E-commerce de productos tecnológicos para la empresa Korvia

Curso: Marcos de Desarrollo Web

Integrantes:

Liendo Teran, David Josue (U24219926)
Duffoó Anamaria, Renzo Paolo (U24217453)
Huayta Huiman, Marco Orlando (U24221284)
Vilca Valle, Cristofer Edwar (U24221943)

Docente: Wenceslao Hooberth Chavez Bedoya

Periodo académico: 2026 – Ciclo 2

ÍNDICE

1. INTRODUCCIÓN 3
2. DESCRIPCIÓN DEL PROBLEMA 3
2.1 Organización o contexto 3
2.2 Situación problemática 3
2.3 Propuesta de solución 4
3. OBJETIVOS DEL PROYECTO 4
3.1 Objetivo general 4
3.2 Objetivos específicos 4
4. ALCANCE DEL PROYECTO 5
4.1 Funcionalidades principales 5
4.2 Funcionalidades no incluidas 6
5. DESARROLLO DEL PROYECTO POR ENTREGAS 6
5.1 Entrega 1 – APF1: Frameworks de Frontend 6
5.1.1 Tecnologías utilizadas 6
5.1.2 Elementos implementados 7
5.1.3 Evidencias 7
5.2 Entrega 2 – APF2: Introducción a Spring 8
5.2.1 Componentes implementados 8
5.2.2 Correcciones de la versión 9
5.2.3 Evidencias 9
5.3 Entrega 3 – APF3: Spring con Base de Datos 10
5.3.1 Configuración de MySQL 10
5.3.2 Entidades JPA 11
5.3.3 Repositorios, servicios y controladores 11
5.3.4 Modelo de base de datos 12
5.3.5 CRUD de categorías y productos 14
5.3.6 Consultas multitabla y DTO 18
5.3.7 Selects en cascada 19
5.3.8 Validaciones 21
5.3.9 Stock, reservas y transacciones 23
5.3.10 Pruebas 25

5.3.11 Resumen de evidencias 25
5.4 Entrega 4 – Proyecto Final: Spring Security 25
6. ARQUITECTURA DEL SISTEMA 26
7. TECNOLOGÍAS UTILIZADAS 26
8. PRUEBAS DEL SISTEMA 27
8.1 Pruebas automatizadas 27
8.2 Pruebas manuales 28
8.3 Resultados 29
9. CONCLUSIONES 29
10. RECOMENDACIONES 30
11. ANEXOS 30
11.1 Modelo entidad-relación 30
11.2 Script de la base de datos 31
11.3 Repositorio y ejecución 32
11.4 Credenciales de prueba 32
11.5 Evidencias complementarias 32

1. INTRODUCCIÓN

Korvia es una empresa minorista dedicada a la comercialización de equipos tecnológicos, especialmente laptops, computadoras de escritorio, equipos all-in-one y componentes. Su operación actual se apoya en hojas de cálculo y registros manuales que no están integrados entre sí, lo que dificulta el control del inventario, la atención de pedidos y la trazabilidad de las reservas.

En los avances anteriores se construyó la interfaz web con HTML, CSS, JavaScript y Bootstrap, y luego se migró la aplicación a Spring Boot, Spring Web y Thymeleaf. En este tercer avance se incorporó **MySQL** como base de datos relacional y **Spring Data JPA / Hibernate** como capa de persistencia para las tablas `roles`, `usuarios`, `categorias`, `productos` y `reservas`. Además, se implementaron CRUD de categorías y productos, consultas multitabla con DTO, selects en cascada, validación de formularios y pruebas automatizadas con H2.

El presente informe documenta el estado del Tercer Avance (APF3). El código fuente, el script `database/korvia.sql`, las capturas y las instrucciones de ejecución se encuentran en el repositorio del proyecto. La seguridad con Spring Security y JWT se identifica como trabajo futuro y no forma parte del alcance de esta entrega.

2. DESCRIPCIÓN DEL PROBLEMA

2.1 Organización o contexto

Korvia atiende a clientes finales y profesionales que buscan equipos de cómputo con asesoría técnica. Su catálogo incluye laptops para estudio y gaming, computadoras de escritorio, estaciones de trabajo, equipos all-in-one y componentes como memoria RAM, discos SSD, gabinetes y kits de actualización.

La empresa no cuenta con una plataforma digital propia para exhibir el catálogo ni con un sistema centralizado que relacione productos, categorías, usuarios y reservas. La información comercial se mantiene en archivos separados, lo que limita la consulta oportuna y la toma de decisiones.

2.2 Situación problemática

La gestión manual de ventas, stock y clientes genera inconsistencias entre lo que se ofrece y lo que realmente existe en almacén. Al no existir una base de datos centralizada, el inventario no se actualiza en tiempo real y no es posible saber con certeza cuántas unidades están disponibles, cuántas están reservadas o cuántas se vendieron.

Además, la ausencia de un modelo de datos estructurado impide escalar la operación, fragmenta el historial de pedidos y dificulta aplicar reglas de negocio como bloquear la eliminación de una categoría con productos o evitar reservas que superen el stock disponible.

2.3 Propuesta de solución

Se implementó una aplicación web con arquitectura MVC que centraliza el catálogo, los usuarios, las categorías, los productos y las reservas en MySQL. La capa de servicios aplica las reglas de negocio y las transacciones; la capa de repositorios utiliza Spring Data JPA para acceder a los datos; y las vistas Thymeleaf muestran la información al cliente y al administrador.

La solución incorpora CRUD de categorías y productos, consultas multitabla proyectadas a DTO, selects en cascada para filtrar productos y reservas, validación del lado del servidor y control transaccional del stock. Con ello se reemplaza el registro manual por una base relacional consistente y verificable.

3. OBJETIVOS DEL PROYECTO

3.1 Objetivo general

Implementar la persistencia de la aplicación Korvia en una base de datos MySQL mediante Spring Data JPA e Hibernate, manteniendo la arquitectura MVC y garantizando la integridad de las categorías, los productos, los usuarios y las reservas.

3.2 Objetivos específicos

- Diseñar un modelo relacional con cinco tablas relacionadas: `roles`, `usuarios`, `categorias`, `productos` y `reservas`.
- Implementar las entidades JPA y los repositorios de acceso a datos con Spring Data JPA.
- Desarrollar los módulos CRUD completos de categorías y productos, con reglas de eliminación segura.
- Implementar consultas multitabla con proyección a DTO para el panel de reportes.
- Implementar selects en cascada en las páginas `/reservas` y `/admin/reportes` mediante endpoints JSON y `fetch`.
- Validar los formularios con Jakarta Bean Validation, `@Valid`, `BindingResult` y mensajes visibles en Thymeleaf.

- Mantener la consistencia del stock mediante operaciones transaccionales de reserva, cancelación y checkout.
- Verificar la persistencia y las reglas de negocio con pruebas automatizadas sobre H2 y pruebas funcionales sobre MySQL.

Spring Security y JWT se consideran trabajo futuro. No se declaran como implementados en este avance.

4. ALCANCE DEL PROYECTO

El sistema contempla dos perfiles de usuario, Cliente y Administrador, y un conjunto de funcionalidades transaccionales y de control orientadas a la gestión comercial de Korvia. El alcance de este avance se concentra en la persistencia relacional, los CRUD, las consultas DTO, los selects en cascada, la validación y las pruebas.

4.1 Funcionalidades principales

Cliente

- Catálogo categorizado con búsqueda, ficha técnica y alertas visuales de disponibilidad.
- Carrito de compras por sesión HTTP, con actualización de cantidades, validación de stock y cálculo de subtotal, IGV y total.
- Reservas temporales de productos por 24 horas, con cancelación y liberación de unidades reservadas.
- Solicitud de cotizaciones y generación de documentos PDF con OpenPDF.
- Checkout con generación de boleta y descarga en PDF.
- Registro e inicio de sesión mediante sesión HTTP.

Administrador

- Panel de administración con métricas de productos, stock crítico, reservas y cotizaciones.
- CRUD completo de categorías, con conteo de productos y bloqueo de eliminación cuando existen productos asociados.
- CRUD completo de productos, con validación de stock y bloqueo de eliminación cuando existen reservas.
- Reportes con consultas multitabla y proyección a DTO.
- Filtros en cascada de categoría, producto y reservas.

Aspectos técnicos

- Persistencia de cinco tablas en MySQL: `roles`, `usuarios`, `categorias`, `productos` y `reservas`.

- Mapeo objeto-relacional con JPA/Hibernate y repositorios Spring Data JPA.
- Validación del lado del servidor con Jakarta Bean Validation.
- Pruebas automatizadas con H2 y verificación funcional sobre MySQL.

4.2 Funcionalidades no incluidas

MySQL y JPA sí forman parte de este avance; la persistencia relacional es el eje central de APF3. Las siguientes funcionalidades quedan fuera del alcance o se consideran trabajo futuro:

- Autenticación y autorización formal con Spring Security, roles protegidos en el servidor y tokens JWT. El login actual utiliza `HttpSession` y guarda el identificador del usuario.
- Integración con una pasarela de pagos externa real, como Stripe o Culqi.
- Módulo de rastreo y tracking logístico de envíos.
- Persistencia de boletas y cotizaciones: en esta entrega se generan en memoria y se exportan a PDF.
- Sincronización del carrito entre dispositivos: el carrito es independiente por sesión HTTP.

5. DESARROLLO DEL PROYECTO POR ENTREGAS

El proyecto se construyó de forma incremental. Las entregas 1 y 2 se conservan como antecedentes del frontend y de la migración a Spring; la entrega 3 concentra la integración con MySQL, la persistencia y las funcionalidades evaluadas en este avance.

5.1 Entrega 1 – APF1: Frameworks de Frontend

En la primera entrega se desarrolló la interfaz de usuario de Korvia con HTML, CSS, JavaScript y Bootstrap. Se construyó la estructura base del sitio, incluyendo la barra de navegación, el diseño responsivo con contenedores, filas y columnas, formularios de registro e inicio de sesión, tablas de productos, imágenes, íconos y ventanas modales. Esta entrega estableció las bases visuales sobre las que se integró posteriormente el backend.

5.1.1 Tecnologías utilizadas

- HTML5 para la estructura de las vistas.
- CSS3 para los estilos personalizados.
- JavaScript para interacciones del lado del cliente.
- Bootstrap 5.3 para el sistema de grillas, componentes y diseño responsivo.

5.1.2 Elementos implementados

- Barra de navegación y menú responsivo.
- Contenedores, filas y columnas de Bootstrap.
- Formularios de registro, login y contacto.
- Tablas y tarjetas para el catálogo.
- Imágenes, íconos y ventanas modales.
- Páginas de inicio, catálogo y detalle de producto.

5.1.3 Evidencias



Figura 1. Página de inicio de Korvia. Fuente: captura del sistema.

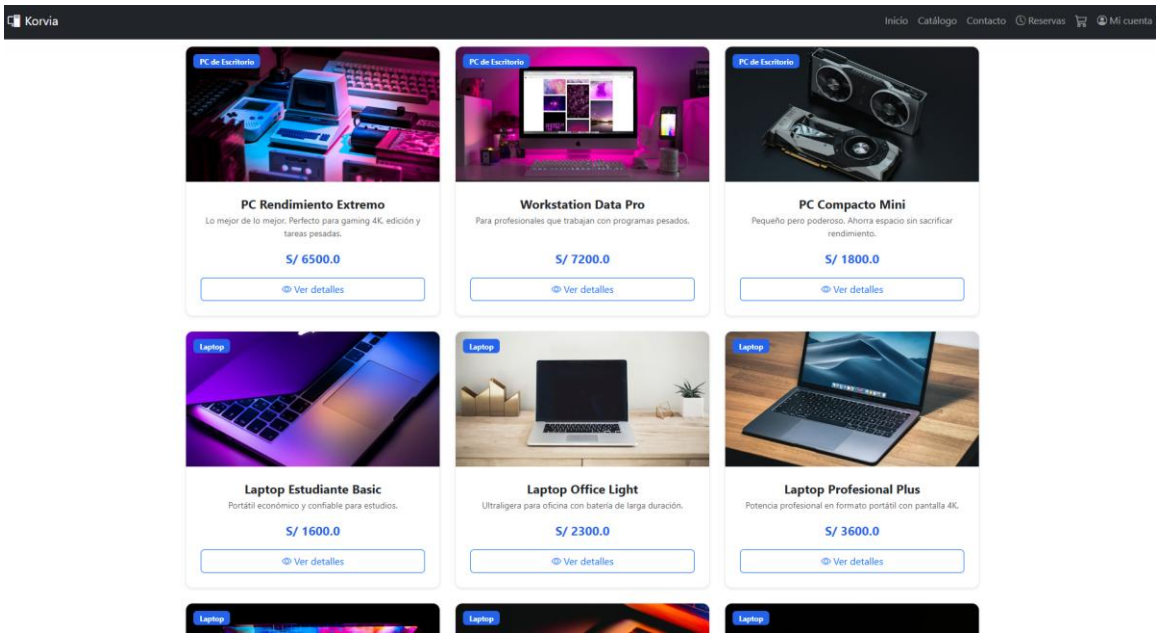


Figura 2. Catálogo de productos con tarjetas y filtros. Fuente: captura del sistema.

5.2 Entrega 2 – APF2: Introducción a Spring

En la segunda entrega se migró la aplicación a Spring Boot, Spring Web y Thymeleaf. Se incorporaron controladores, servicios, modelos, vistas dinámicas y fragmentos reutilizables. La navegación y la sesión del usuario se gestionaron con `HttpSession` y un `@ControllerAdvice` que inyecta el usuario actual en todas las vistas.

En APF2 los servicios de productos y usuarios todavía trabajaban con colecciones en memoria. Esa situación se corrigió en APF3 con la incorporación de JPA/Hibernate y repositorios. Las vistas Thymeleaf se ubican en `src/main/resources/templates`; la generación de PDF se realiza con OpenPDF; y el JavaScript activo está incorporado en las plantillas, mientras que `script.js` es un archivo legado que no se utiliza.

5.2.1 Componentes implementados

- Controladores MVC para inicio, catálogo, carrito, checkout, boletas, cotizaciones, reservas, login, registro y contacto.
- Servicios de lógica de negocio para productos, carrito, boletas, cotizaciones, reservas y usuarios.
- Modelos de dominio y entidades del sistema.
- Vistas Thymeleaf y fragmentos reutilizables (`head`, `navbar`, `footer` y `modal-cuenta`).
- Navegación dinámica y sesión con `HttpSession`.
- Recursos estáticos organizados en `static/css`, `static/js` y `static/img`.

5.2.2 Correcciones de la versión

- Las vistas se encuentran en `templates`, no en `static`.
- La generación de PDF utiliza OpenPDF 1.3.30, no iText.
- El JavaScript activo está dentro de las plantillas; `script.js` no es el archivo que gobierna la interfaz actual.
- Los servicios de productos y usuarios que en APF2 trabajaban en memoria fueron reemplazados por repositorios JPA en APF3.

5.2.3 Evidencias

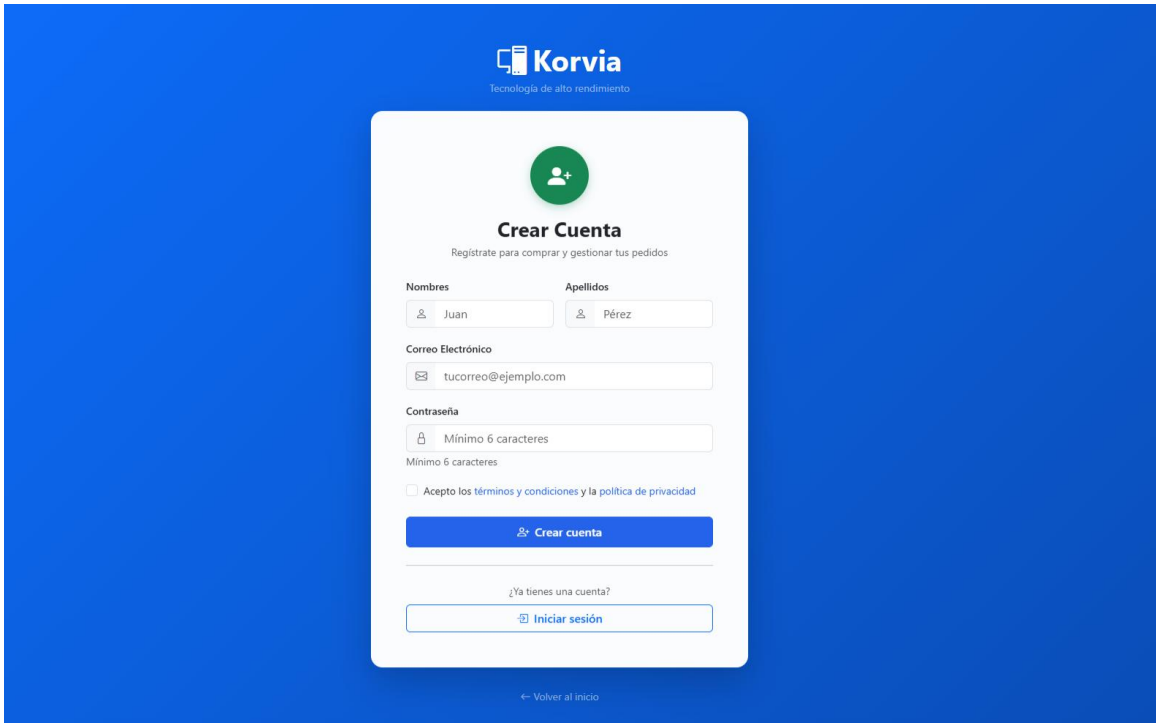


Figura 3. Formulario de creación de cuenta. Fuente: captura del sistema.

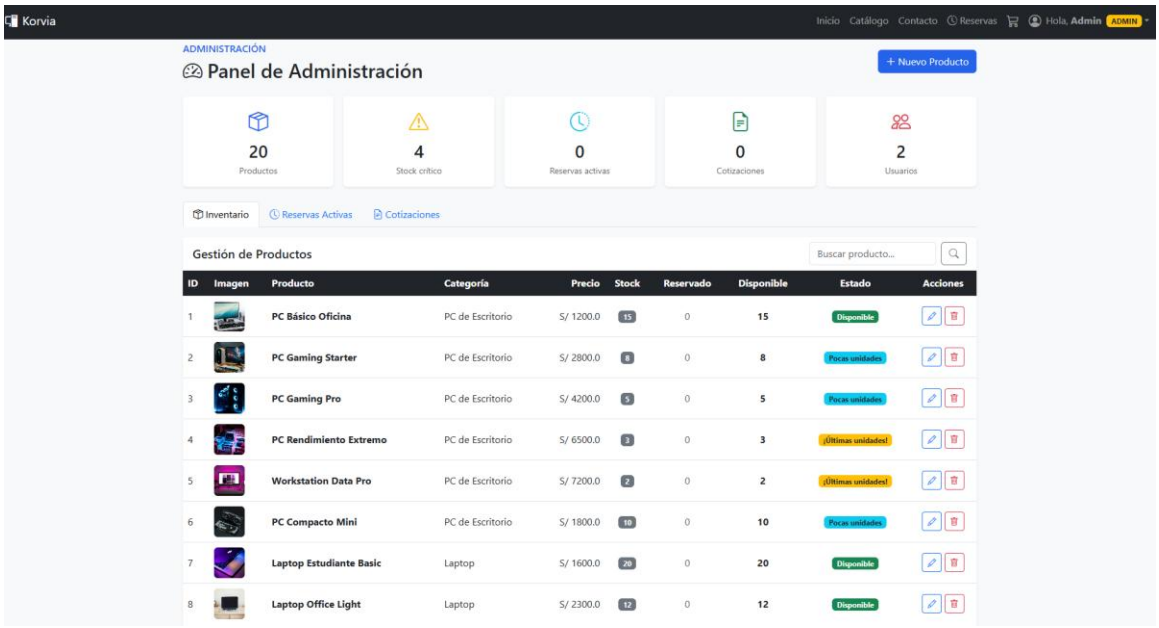


Figura 4. Panel de administración de APF2. Fuente: captura del sistema.

5.3 Entrega 3 – APF3: Spring con Base de Datos

La tercera entrega traslada la información de memoria a MySQL. Se crearon cinco tablas relacionadas, cinco entidades JPA, cinco repositorios, servicios transaccionales y controladores para los CRUD y los reportes. La aplicación conserva el patrón MVC y ahora obtiene los datos desde la base relacional mediante Hibernate.

Esta sección describe la configuración de la conexión, el modelo de datos, los módulos CRUD, las consultas DTO, los selects en cascada, las validaciones, la lógica de stock y las pruebas que respaldan la entrega.

5.3.1 Configuración de MySQL

La conexión se configura en `src/main/resources/application.properties`. Spring Boot utiliza el driver JDBC de MySQL y HikariCP como pool de conexiones. La contraseña se lee de la variable de entorno `DB_PASSWORD` y no se escribe en el repositorio. Hibernate administra el esquema con `ddl-auto=update`; el script `database/korvia.sql` queda como entregable y permite crear la base desde cero.

```
spring.datasource.url=jdbc:mysql://localhost:3306/korvia?useSSL=false&allowPublicKeyRetrie
val=true&serverTimezone=America/Lima&characterEncoding=UTF-8
spring.datasource.username=root
spring.datasource.password=${DB_PASSWORD:}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
spring.sql.init.mode=never
```

La contraseña local se define en la terminal antes de arrancar: $env:DB_PASSWORD='tu_password_local'. No debe incluirse en el informe ni en el repositorio.

5.3.2 Entidades JPA

Las clases `Rol`, `Usuario`, `Categoria`, `Producto` y `Reserva` están anotadas con `@Entity` y `@Table`. Usan `@Id` y `@GeneratedValue` para las claves primarias, `@Column` para las columnas y `@ManyToOne` / `@OneToMany` para las relaciones. `Producto` mantiene la categoría como relación obligatoria y `Reserva` relaciona el producto y, opcionalmente, el usuario.

```
@Entity
@Table(name = "productos")
public class Producto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    @JoinColumn(name = "categoria_id", nullable = false)
    private Categoria categoria;

    @Column(nullable = false)
    private Integer stock;

    @Column(nullable = false)
    private Integer reservado = 0;
}
```

La entidad `Reserva` agrega `producto_id` como clave foránea obligatoria y `usuario_id` como clave foránea opcional. Además almacena el nombre y correo del cliente, la cantidad, las fechas de reserva y expiración, y el estado (`ACTIVA`, `EXPIRADA` o `CANCELADA`).

5.3.3 Repositorios, servicios y controladores

Se crearon cinco interfaces que extienden `JpaRepository`: `RolRepository`, `UsuarioRepository`, `CategoriaRepository`, `ProductoRepository` y `ReservaRepository`. Los servicios concentran las reglas de negocio y las transacciones; los controladores reciben las peticiones HTTP y devuelven vistas Thymeleaf o JSON. La separación evita que la lógica de stock o las validaciones dependan de la capa web.

Tabla 1. Capas de la aplicación y componentes principales.

Capa	Responsabilidad	Componentes principales
Controller	Recibe peticiones HTTP, llena el Model y devuelve vistas o JSON.	CategoriaAdminController, ProductoAdminController, ReporteController, ProductoApiController, ReporteApiController.
Service	Aplica reglas de negocio y transacciones.	CategoriaService, ProductoService, ReservaService, CheckoutService, ReporteService, UsuarioService.
Repository	Acceso a datos con Spring Data JPA y consultas JPQL.	RolRepository, UsuarioRepository, CategoriaRepository, ProductoRepository, ReservaRepository.
Model	Entidades JPA y POJOs de dominio.	Rol, Usuario, Categoria, Producto, Reserva, Boleta, Cotizacion, ItemCarrito.

Capa	Responsabilidad	Componentes principales
DTO	Formularios y proyecciones de consultas.	CategoriaForm, ProductoForm, ReservaForm, CategoriaConteoDTO, ProductoReporteDTO, ReservaDetalleDTO.
Config	Carga inicial de datos de prueba.	DataInitializer.

Las operaciones que modifican stock están anotadas con `@Transactional`. `ReservaService` incrementa `reservado` al reservar y lo libera al cancelar; `CheckoutService` descuenta `stock` y libera las unidades reservadas equivalentes dentro de la misma transacción.

5.3.4 Modelo de base de datos

El modelo relacional está formado por cinco tablas. La tabla `roles` alimenta los perfiles ADMIN y CLIENTE; `usuarios` referencia al rol; `categorias` agrupa el catálogo; `productos` referencia a la categoría y guarda stock y reservado; y `reservas` relaciona un producto con un usuario opcional. El diagrama completo se muestra en la Figura 5.

Tabla 2. Tablas, claves primarias, claves foráneas y propósito.

Tabla	Clave primaria	Claves foráneas	Propósito
`roles`	`id`	—	Catálogo de roles del sistema (ADMIN, CLIENTE).
`usuarios`	`id`	`rol_id` → `roles.id`	Usuarios registrados y su rol.
`categorias`	`id`	—	Categorías del catálogo.
`productos`	`id`	`categoria_id` → `categorias.id`	Productos, precio, stock, reservado y especificaciones.
`reservas`	`id`	`producto_id` → `productos.id`; `usuario_id` → `usuarios.id` (nullable)	Reservas temporales con fecha de expiración y estado.

Tabla 3. Relaciones entre entidades y reglas de negocio.

Relación	Cardinalidad	Implementación JPA	Regla de negocio
`roles` → `usuarios`	1:N	`Usuario.rol` con `@ManyToOne`	Todo usuario debe tener un rol asignado.
`categorias` → `productos`	1:N	`Producto.categoria` con `@ManyToOne`	Un producto pertenece a una sola categoría.
`productos` → `reservas`	1:N	`Reserva.producto` con `@ManyToOne`	Toda reserva está vinculada a un producto existente.
`usuarios` → `reservas`	1:N opcional	`Reserva.usuario` con `@ManyToOne` y `nullable`	La reserva puede ser anónima; el nombre y correo quedan en la reserva.

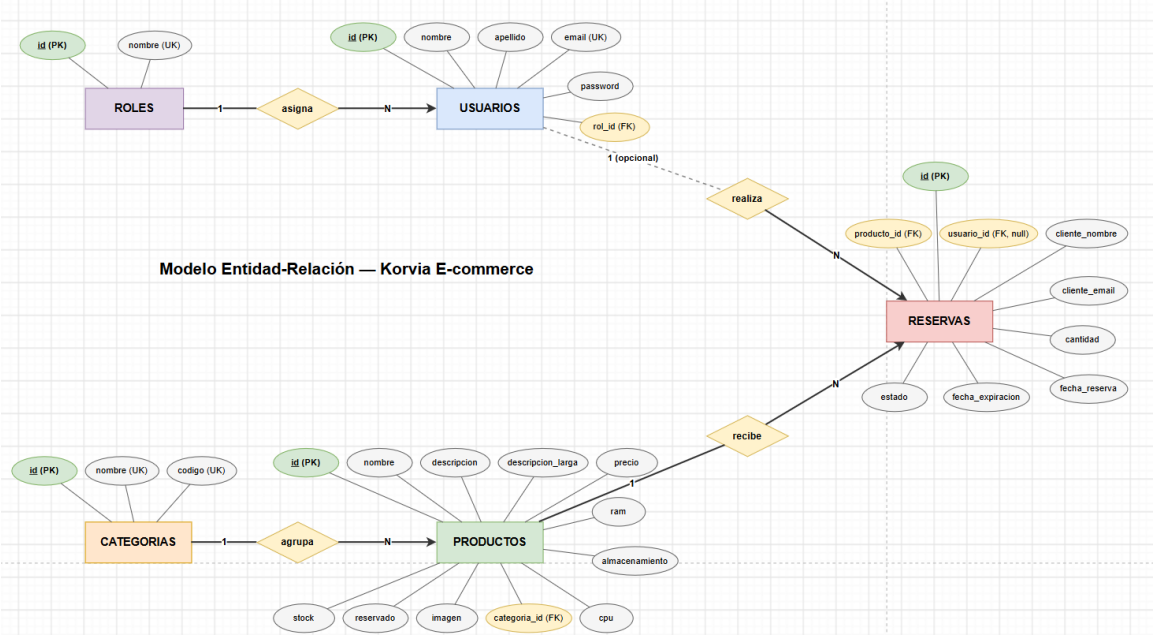


Figura 5. Modelo entidad-relación de la base de datos Korvia. Fuente: elaboración propia.

Evidencia de la base de datos korvia

Salida real de consultas ejecutadas en MySQL 26.7 sobre la base de datos del proyecto.

```
mysql> SHOW TABLES;
```

Tables_in_korvia
categorias
productos
reservas
roles
usuarios

```
mysql> SELECT 'roles' AS tabla, COUNT(*) AS registros FROM roles
    -> UNION ALL SELECT 'usuarios', COUNT(*) FROM usuarios
    -> UNION ALL SELECT 'categorias', COUNT(*) FROM categorias
    -> UNION ALL SELECT 'productos', COUNT(*) FROM productos
    -> UNION ALL SELECT 'reservas', COUNT(*) FROM reservas;
```

tabla	registros
roles	2
usuarios	2
categorias	4
productos	20
reservas	1

```
mysql> SELECT TABLE_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
    -> FROM information_schema.KEY_COLUMN_USAGE
    -> WHERE TABLE_SCHEMA = 'korvia' AND REFERENCED_TABLE_NAME IS NOT NULL;
```

TABLE_NAME	COLUMN_NAME	REFERENCED_TABLE_NAME	REFERENCED_COLUMN_NAME
productos	categoria_id	categorias	id
reservas	producto_id	productos	id
reservas	usuario_id	usuarios	id
usuarios	rol_id	roles	id

```
5 tables in set (0.00 sec)
-- Consultas ejecutadas con --default-character-set=utf8mb4
```

Figura 6. Salida de consultas SQL sobre MySQL: tablas, registros y claves foráneas. Fuente: MySQL 26.7.

El stock se interpreta en tres valores: `stock` es la cantidad total, `reservado` es la cantidad apartada por reservas activas y la disponibilidad se calcula como `stock` -

`reservado`. En la interfaz, el estado del producto se muestra como Disponible, Pocas unidades, Stock crítico o Agotado según ese cálculo.

5.3.5 CRUD de categorías y productos

Se implementaron dos módulos CRUD completos. El de categorías permite listar, crear, editar y eliminar, con validación de código y nombre únicos. El de productos permite listar y buscar en el panel de administración, crear, editar y eliminar, con validación de precio, stock, categoría e imagen. La eliminación de una categoría se bloquea si tiene productos; la de un producto se bloquea si tiene reservas.

Tabla 4. Rutas del CRUD de categorías.

Método	Ruta	Acción
GET	`/admin/categorias`	Listado con conteo de productos, stock total, reservado y disponible.
GET	`/admin/categoria/nuevo`	Formulario de alta.
POST	`/admin/categoria/guardar`	Crear categoría.
GET	`/admin/categoria/editar/{id}`	Formulario de edición con datos cargados.
POST	`/admin/categoria/actualizar/{id}`	Actualizar categoría.
POST	`/admin/categoria/eliminar/{id}`	Eliminar con validación de productos asociados.

Tabla 5. Rutas del CRUD de productos.

Método	Ruta	Acción
GET	`/administracion`	Listado y búsqueda de productos en la pestaña Inventario.
GET	`/admin/producto/nuevo`	Formulario de alta.
POST	`/admin/producto/guardar`	Crear producto.
GET	`/admin/producto/editar/{id}`	Formulario de edición con datos cargados.
POST	`/admin/producto/actualizar/{id}`	Actualizar producto.
POST	`/admin/producto/eliminar/{id}`	Eliminar con validación de reservas asociadas.

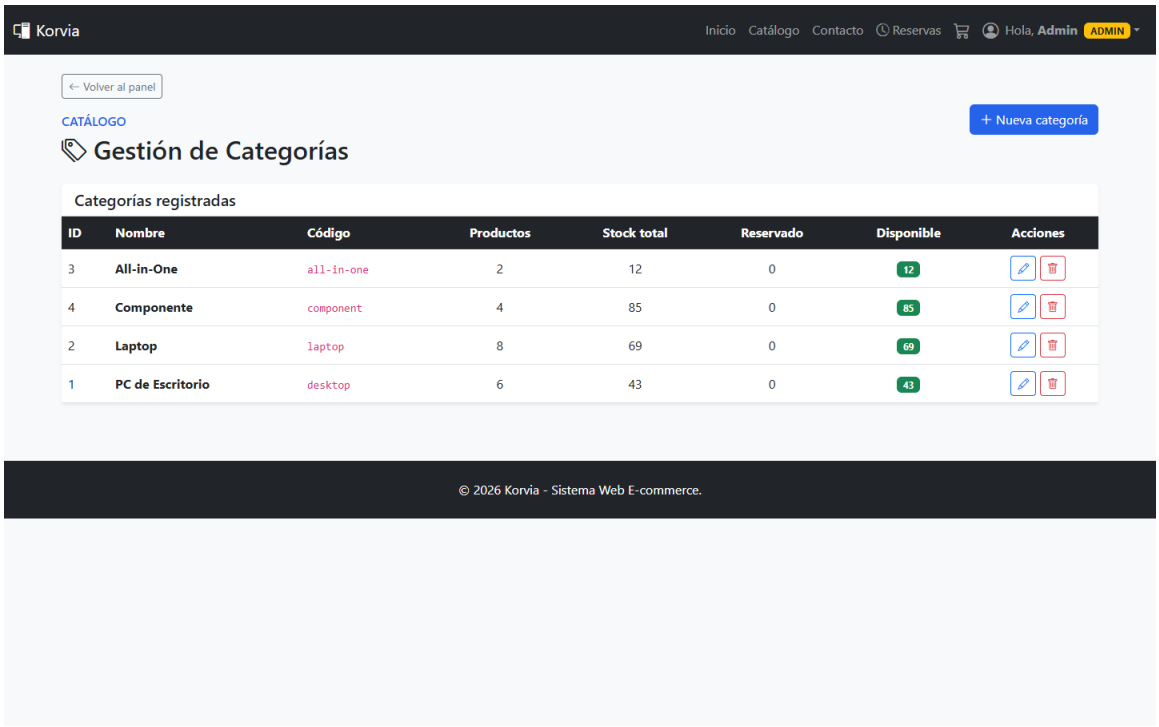


Figura 7. Listado de categorías con conteo de productos y stock. Fuente: captura del sistema.

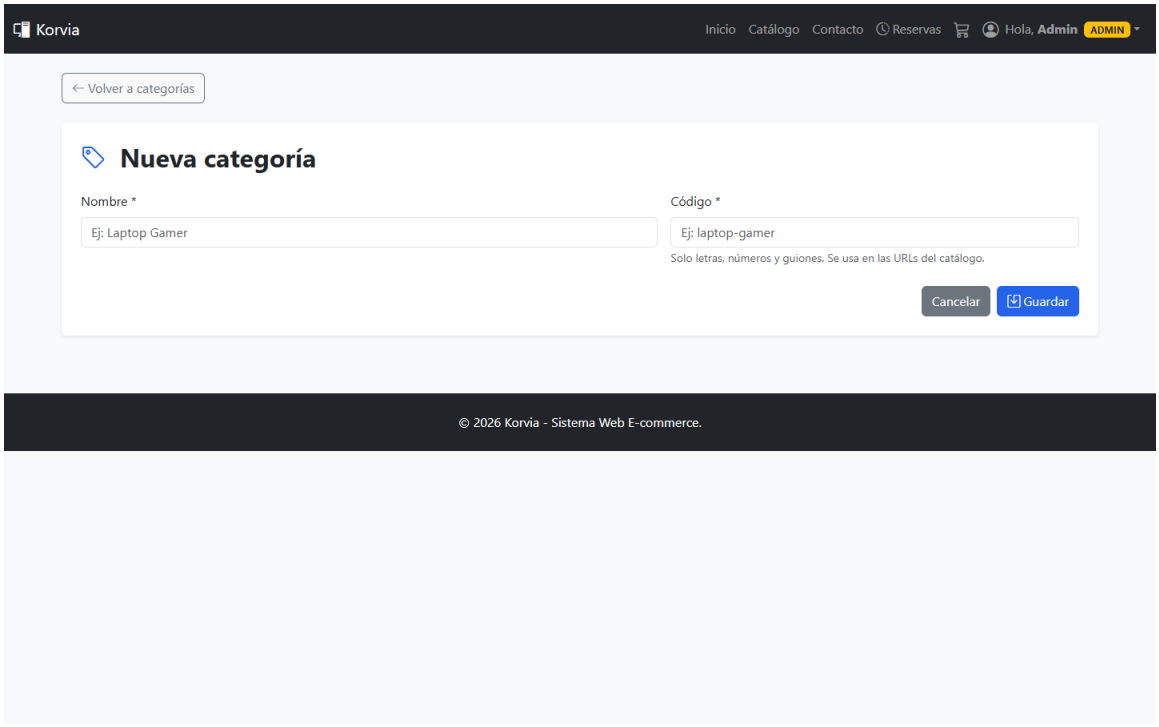


Figura 8. Formulario de alta de categoría. Fuente: captura del sistema.

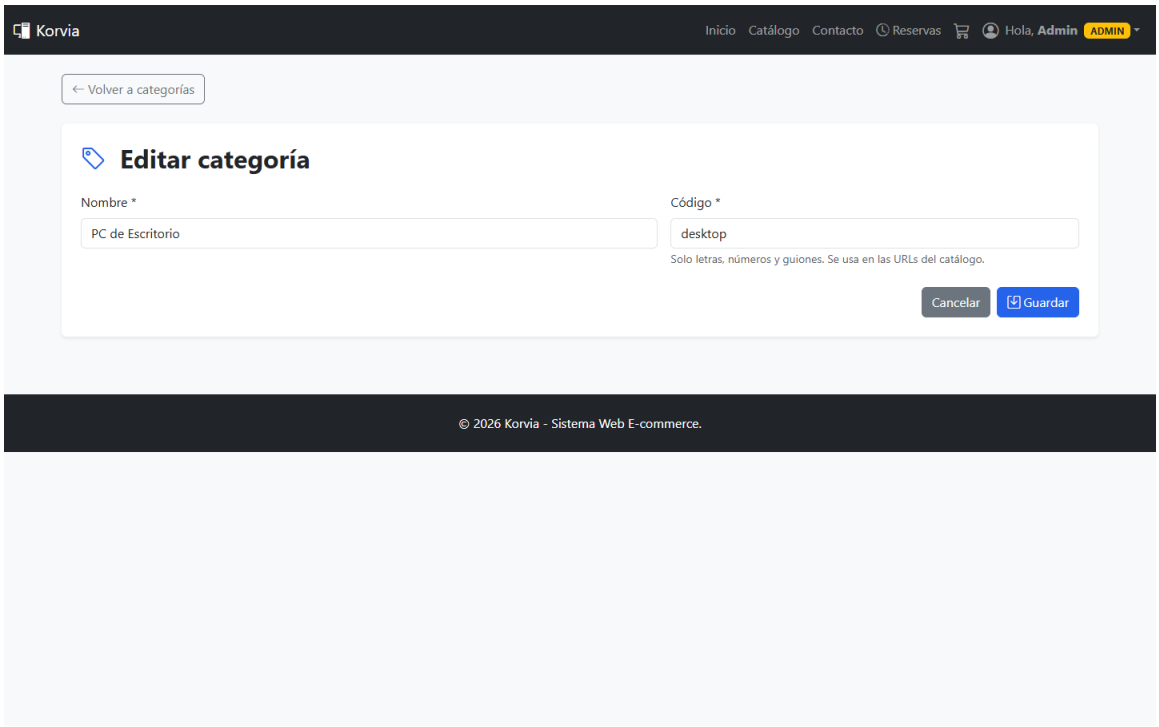


Figura 9. Formulario de edición de categoría. Fuente: captura del sistema.

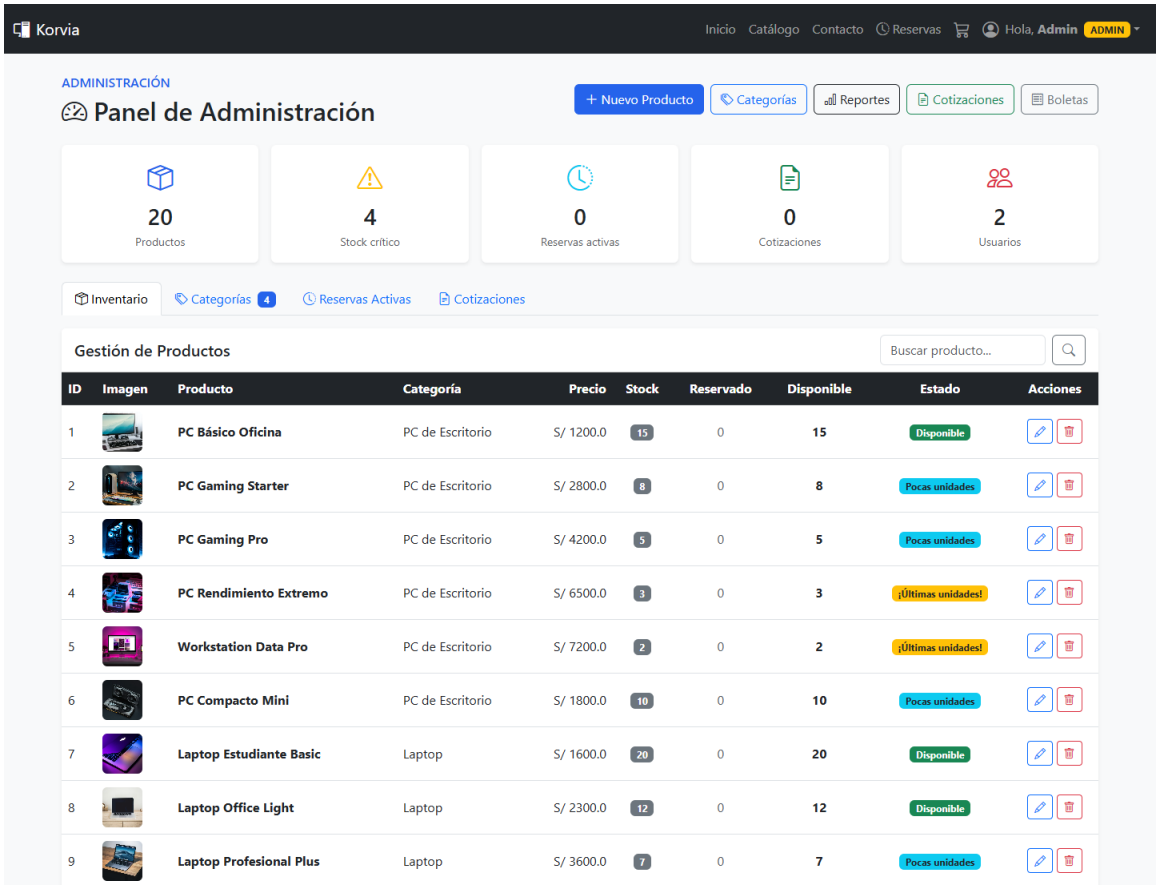


Figura 10. Inventario de productos en el panel de administración. Fuente: captura del sistema.

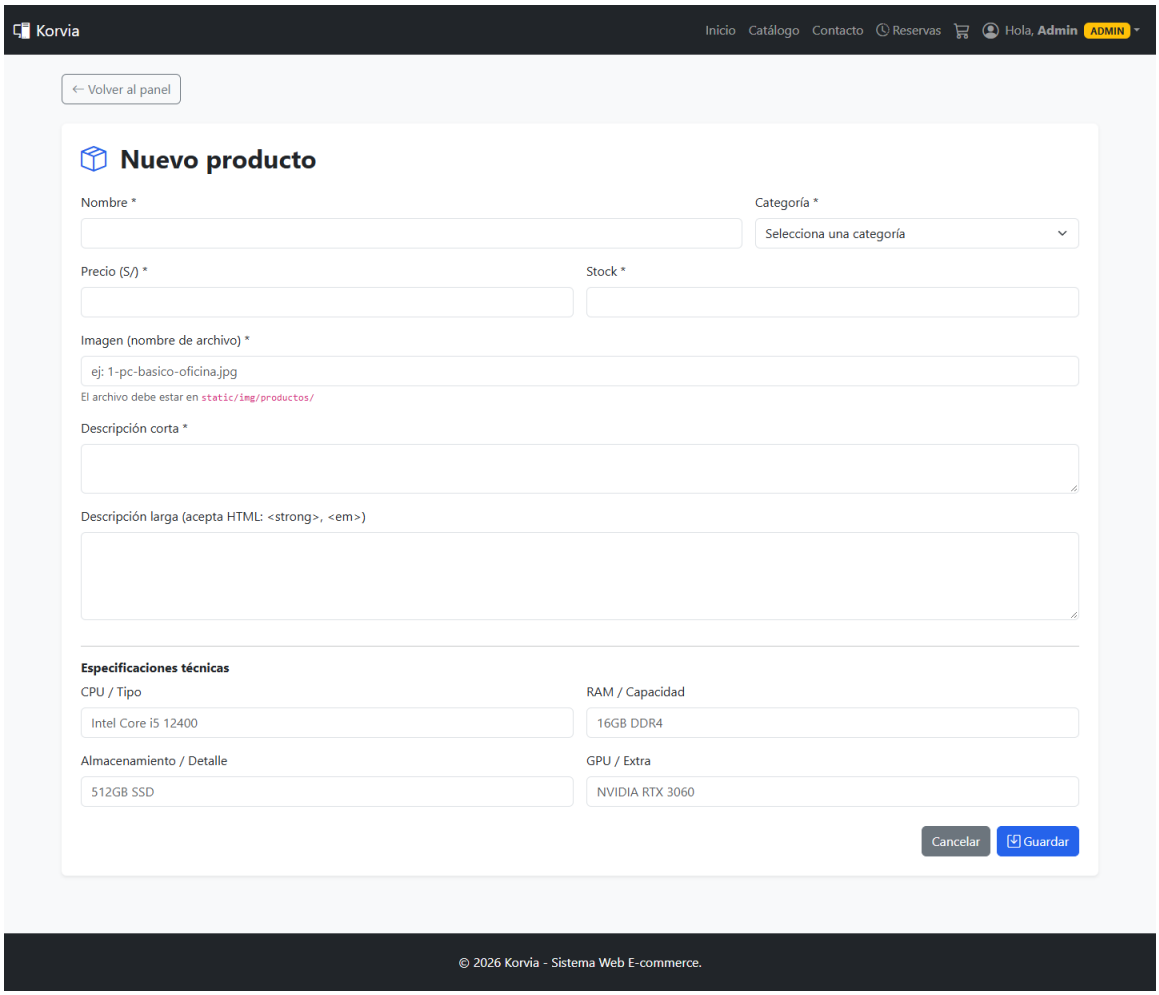


Figura 11. Formulario de alta de producto. Fuente: captura del sistema.

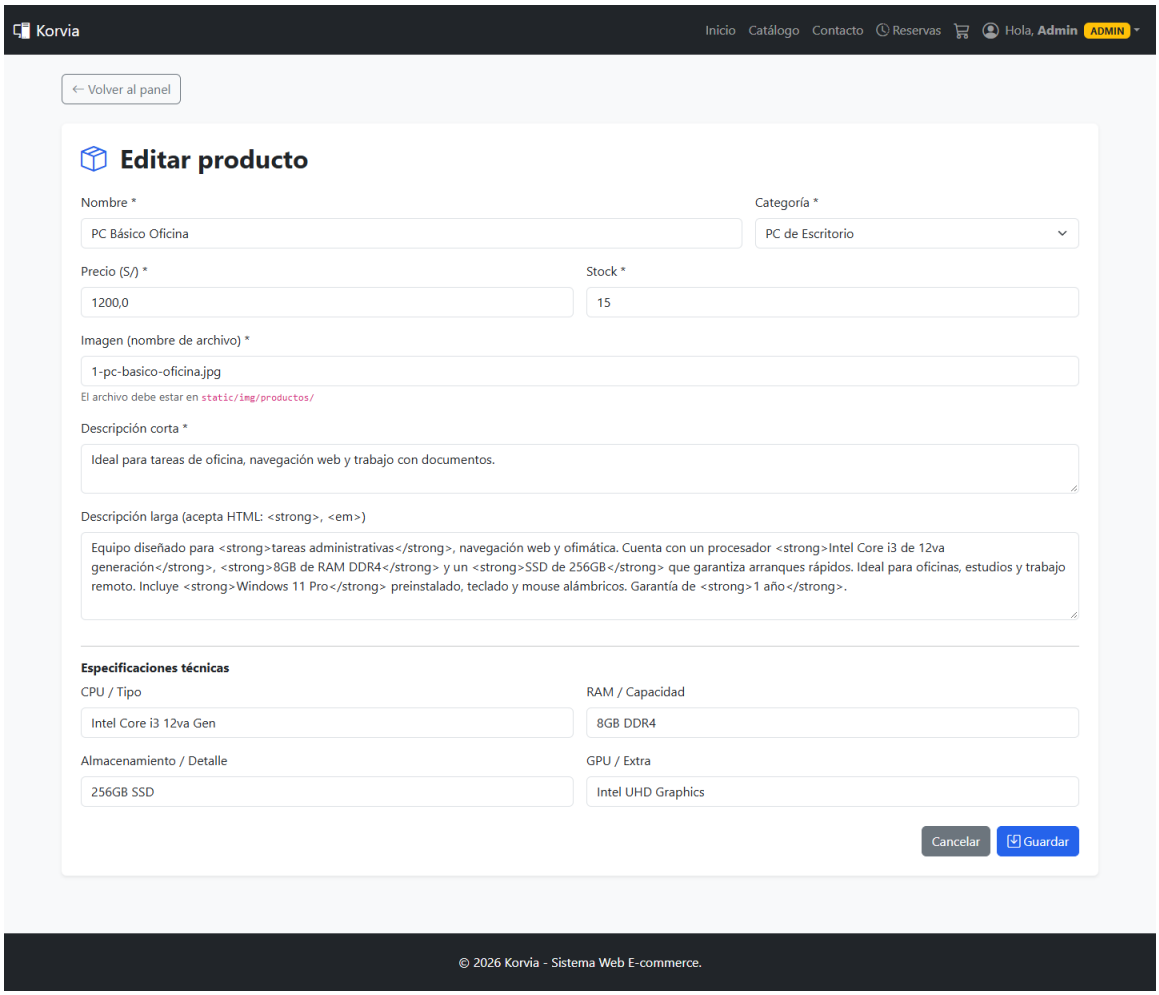


Figura 12. Formulario de edición de producto. Fuente: captura del sistema.

5.3.6 Consultas multitabla y DTO

Los reportes utilizan consultas JPQL que combinan varias tablas y proyectan el resultado directamente a DTO. Esto evita exponer entidades JPA en las vistas y reduce los datos transportados. Se implementaron tres consultas representativas.

Tabla 6. Consultas multitabla y DTO utilizados en los reportes.

DTO	Consulta	Tablas involucradas	Uso
`CategoriaConteoDTO`	`CategoriaRepository.listarConConteo`	`categorias` + `productos` (`LEFT JOIN`)	Conteo de productos, stock total y reservado por categoría.
`ProductoReporteDTO`	`ProductoRepository.listarReporte`	`productos` + `categorias` (`JOIN`)	Inventario detallado con categoría, precio y stock.
`ReservaDetalleDTO`	`ReservaRepository.listarDetalle`	`reservas` + `productos` + `categorias` + `usuarios` (`LEFT JOIN`)	Reporte de reservas con cliente y correo.

```
@Query("""
select new pe.edu.utp.korvia.dto.CategoriaConteoDTO(
    c.id, c.nombre, c.codigo, count(p.id), sum(p.stock), sum(p.reservado))
from Categoria c
left join c.productos p
group by c.id, c.nombre, c.codigo
order by c.nombre asc
""")
List<CategoriaConteoDTO> listarConConteo();
```

La consulta de categorías usa `LEFT JOIN` para incluir categorías sin productos, `COUNT` para contar productos y `SUM` para totalizar stock y reservado. La consulta de productos usa `JOIN` porque todo producto tiene categoría. La consulta de reservas usa `LEFT JOIN` con usuarios para admitir reservas anónimas y `COALESCE` para mostrar el correo del usuario o el correo ingresado en la reserva.

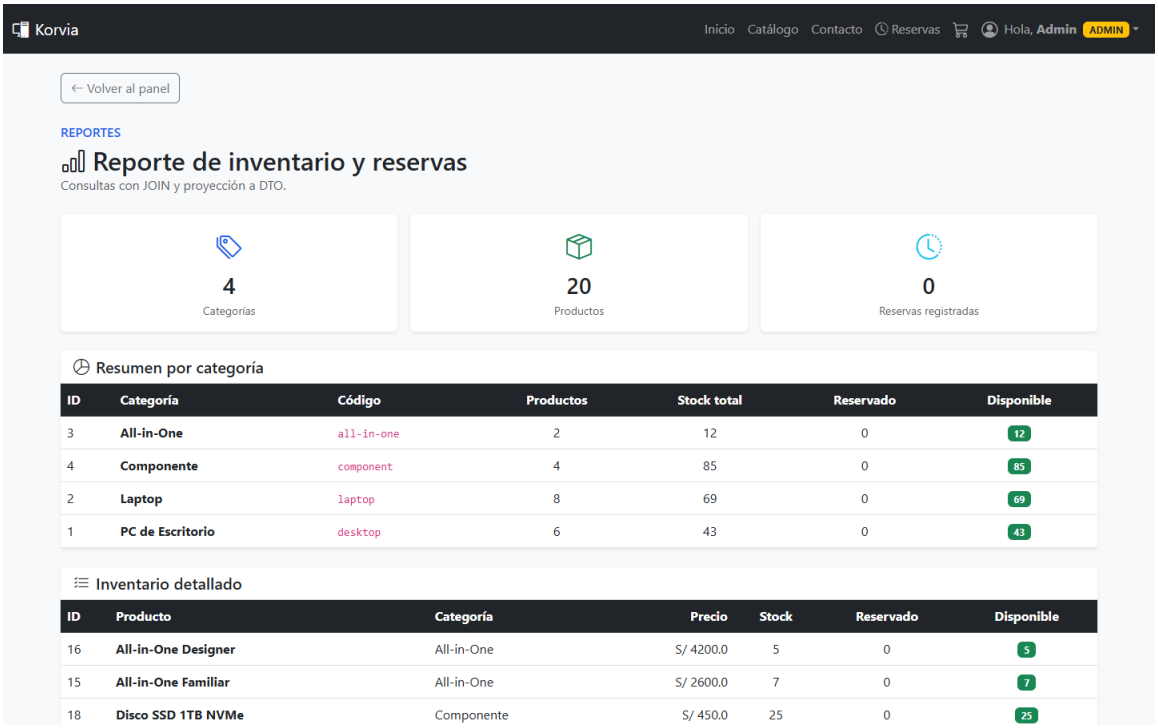


Figura 13. Reporte de inventario y reservas con datos proyectados a DTO. Fuente: captura del sistema.

5.3.7 Selects en cascada

Los selects en cascada se implementaron en dos páginas. En `/reservas`, al seleccionar una categoría se consulta `GET /api/productos?categoriaId=...` y se cargan los productos de esa categoría. En `/admin/reportes`, al seleccionar una categoría se cargan sus productos y, al seleccionar un producto, se consulta `GET /api/reportes/reservas?productoId=...` para filtrar la tabla de reservas.

- `ProductoApiController` expone `GET /api/productos` y devuelve `ProductoOpcionDTO`.
- `ReporteApiController` expone `GET /api/reportes/reservas` y devuelve `ReservaDetalleDTO`.
- La comunicación se realiza con `fetch` y los endpoints devuelven JSON.

- Los selects se actualizan sin recargar la página y solo muestran opciones válidas para la selección previa.

```
const respuesta = await fetch(`/api/productos?categoriaId=${categoriaId}`);
const productos = await respuesta.json();
productoSelect.innerHTML = '<option value="">Selecciona un producto</option>';
productos.forEach(p => {
    productoSelect.add(new Option(`${p.nombre} (disponible: ${p.stock - p.reservado})`, p.
id));
});
```

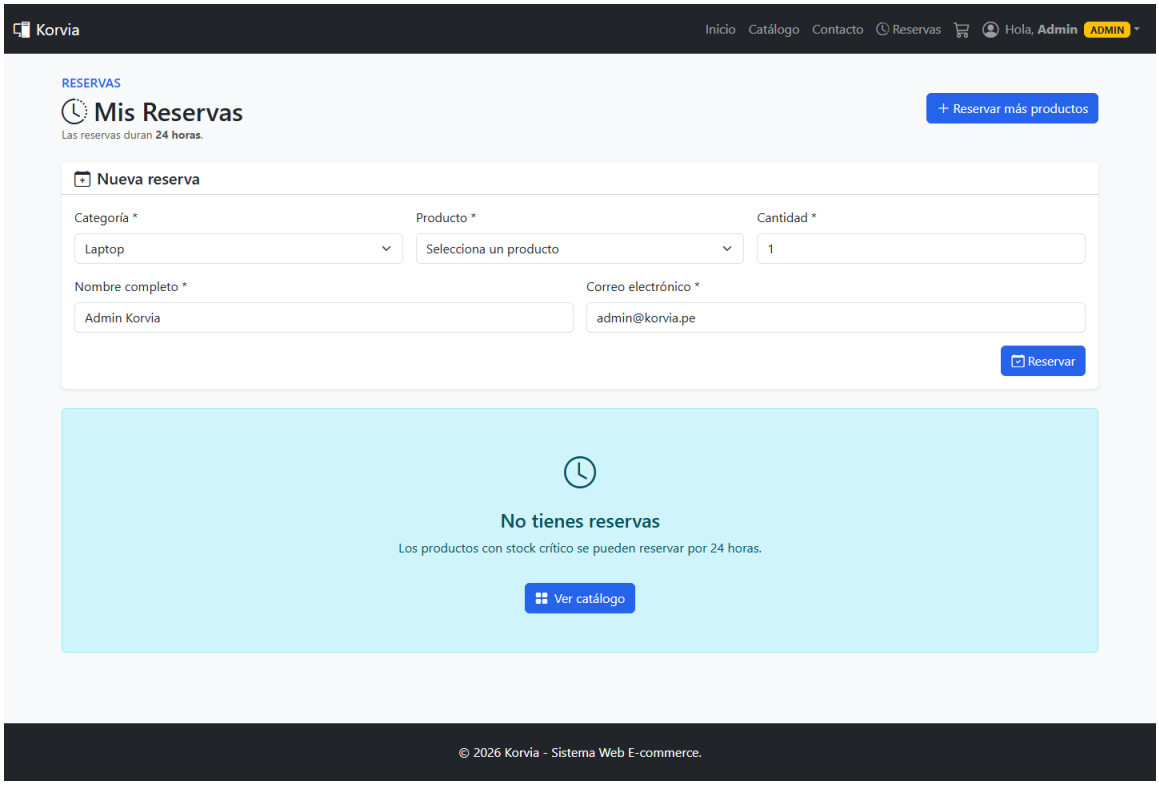


Figura 14. Selects en cascada de categoría y producto en /reservas. Fuente: captura del sistema.

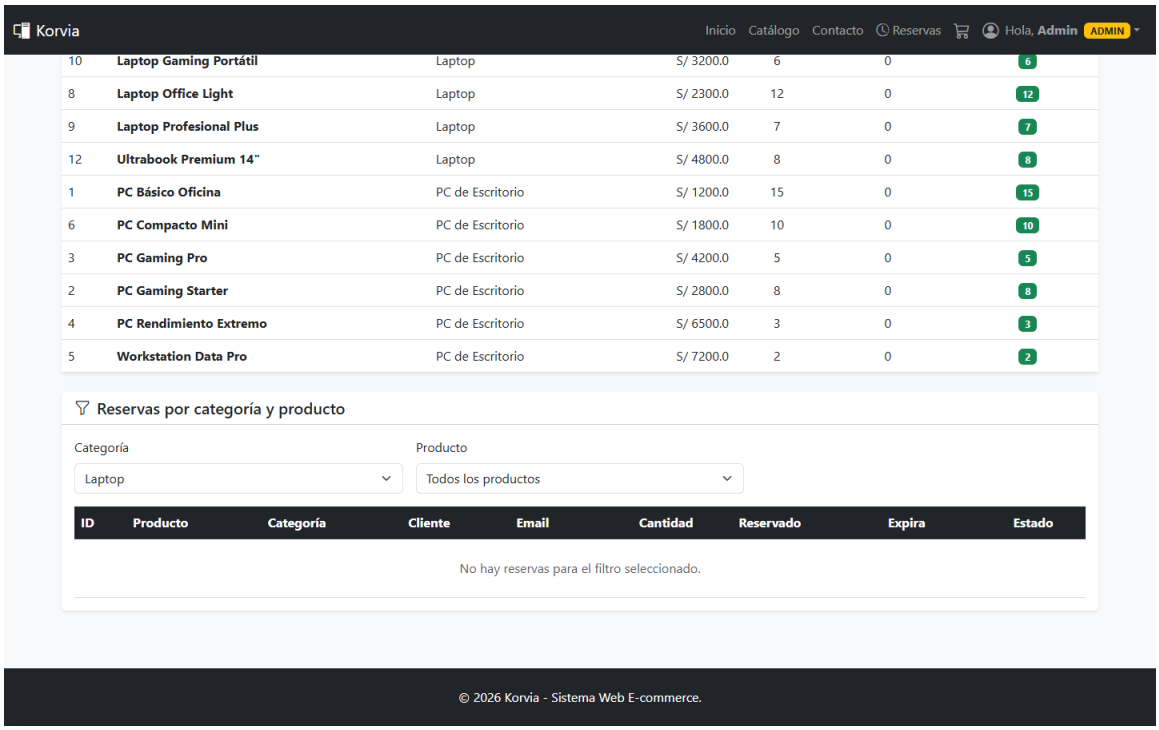


Figura 15. Filtros en cascada de categoría, producto y reservas en /admin/reportes. Fuente: captura del sistema.

5.3.8 Validaciones

Los formularios se validan en el servidor con Jakarta Bean Validation. Los controladores reciben el formulario con `@Valid` y un `BindingResult`; si hay errores, vuelven a mostrar la vista sin guardar. Las plantillas usan `th:errors` para mostrar los mensajes y los formularios incluyen `novalidate` para que la validación del servidor sea visible.

Tabla 7. Anotaciones de validación utilizadas.

Anotación	Uso	Ejemplo en el proyecto
`@NotBlank`	Obliga a que un texto tenga contenido visible.	Nombre y código de categoría.
`@NotNull`	Obliga a que un valor no sea nulo.	Precio, stock y categoría del producto.
`@Size`	Limita la longitud mínima y máxima.	Nombre de producto entre 3 y 200 caracteres.
`@Email`	Valida el formato de correo.	Correo del cliente en una reserva.
`@Min`	Exige un mínimo numérico.	Cantidad de reserva mayor o igual a 1; stock no negativo.
`@DecimalMin`	Exige un mínimo decimal.	Precio mayor a 0.01.
`@Pattern`	Valida una expresión regular.	Código de categoría con letras, números y guiones.

El flujo es: formulario con `novalidate` → controlador con `@Valid` y `BindingResult` → vista con `th:errors` → mensaje visible sin perder los datos ingresados. Las reglas de

unicidad y las validaciones de negocio se aplican además en los servicios, por ejemplo al comprobar si ya existe un código de categoría o si el stock es suficiente para una reserva.

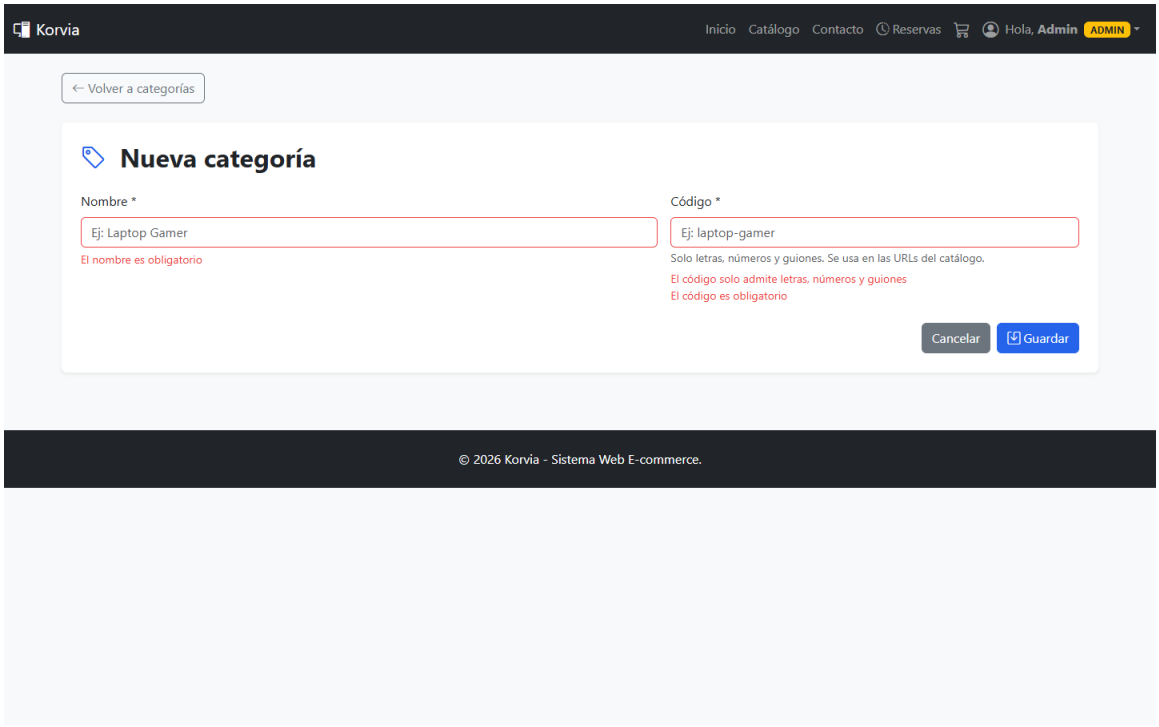


Figura 16. Mensajes de validación en el formulario de categoría. Fuente: captura del sistema.

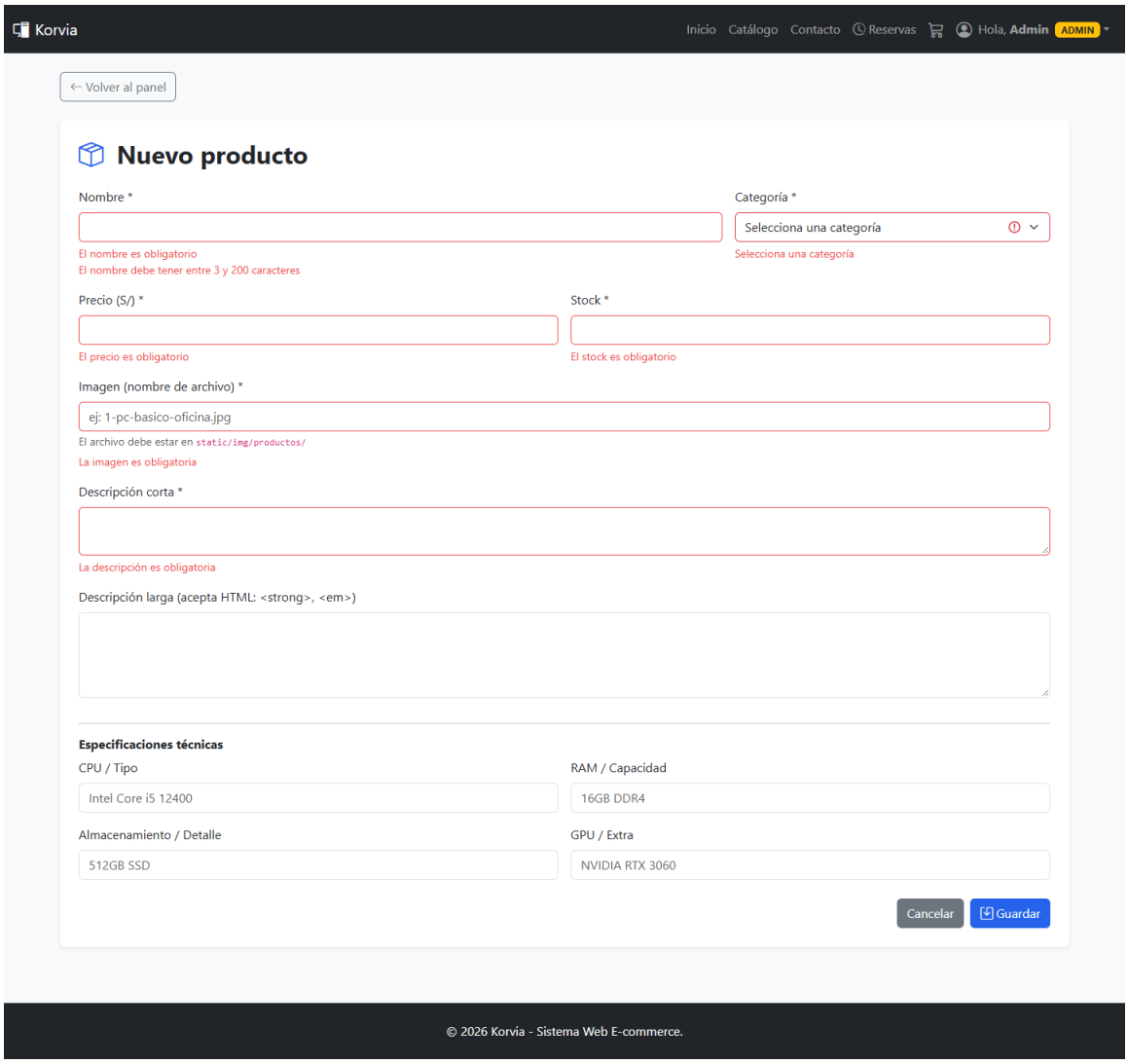


Figura 17. Mensajes de validación en el formulario de producto. Fuente: captura del sistema.

5.3.9 Stock, reservas y transacciones

La reserva incrementa `reservado` y reduce la disponibilidad sin descontar todavía el `stock` total. La cancelación libera la cantidad reservada. El checkout descuenta el stock de los productos del carrito y libera las unidades reservadas equivalentes para no dejar `reservado` inflado después de la compra. Todas estas operaciones se ejecutan con `@Transactional`.

En la prueba funcional se reservó una unidad de Laptop Estudiante Basic. El producto tenía stock 20 y reservado 0; después de la reserva quedó con reservado 1 y disponible 19. La reserva se conservó después de reiniciar la aplicación, lo que confirma la persistencia en MySQL.

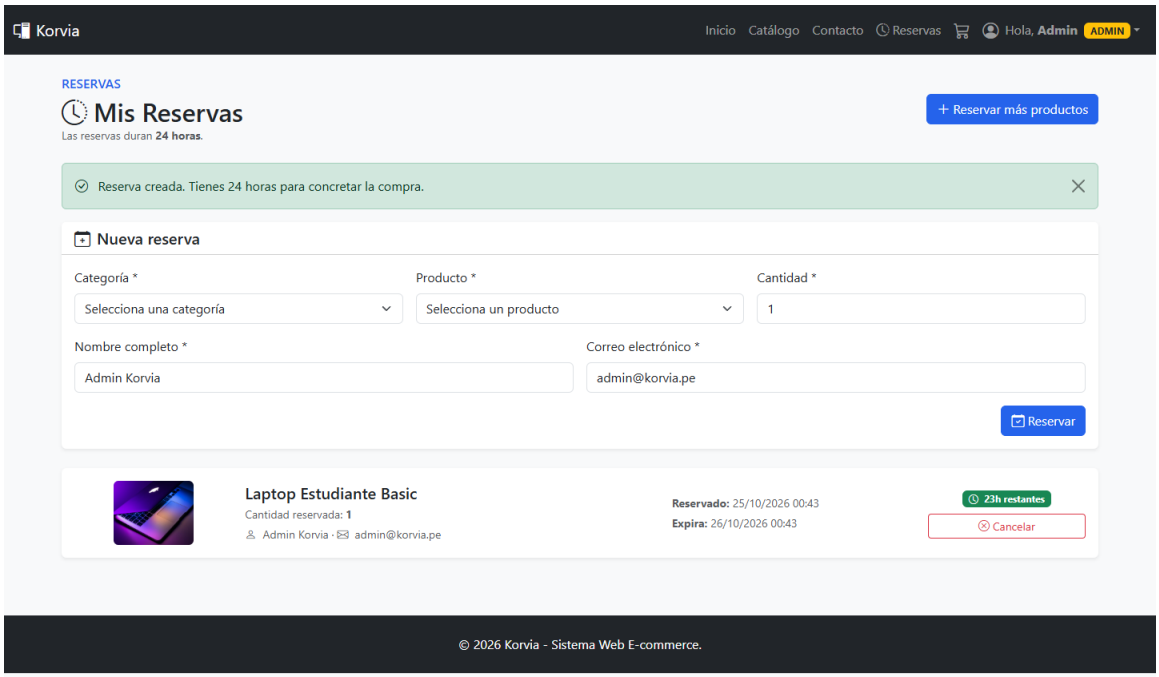


Figura 18. Reserva creada con vigencia de 24 horas. Fuente: captura del sistema.

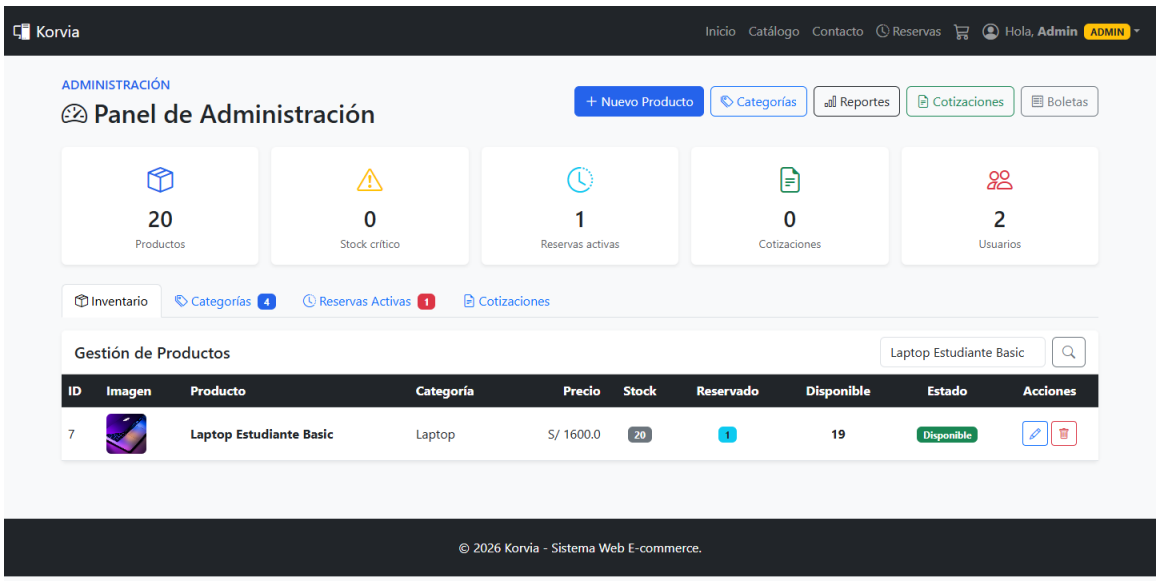


Figura 19. Stock actualizado: 20 unidades totales, 1 reservada y 19 disponibles. Fuente: captura del sistema.

5.3.10 Pruebas

Las pruebas automatizadas usan H2 en memoria con `ddl-auto=create-drop`, por lo que no dependen de MySQL. Se ejecutan con Maven Wrapper y verifican el contexto, los datos iniciales, los DTO, los CRUD y la reserva de stock.

```
.\mvnw.cmd clean test

Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Tabla 8. Pruebas automatizadas ejecutadas.

N.º	Prueba	Propósito	Resultado
1	`KorviaApplicationTests#contextLoads`	Verificar que el contexto Spring arranque.	Correcto
2	`KorviaPersistenceTests#cargaDatosInicialesYConsultaDto`	Validar datos iniciales, login y consultas DTO.	Correcto
3	`KorviaPersistenceTests#crudCategoriaFunciona`	Crear, actualizar, consultar y eliminar una categoría.	Correcto
4	`KorviaPersistenceTests#crudProductoFunciona`	Crear, actualizar, consultar y eliminar un producto.	Correcto
5	`KorviaPersistenceTests#reservaIncrementaYLiberaStockReservado`	Comprobar incremento y liberación de reservado.	Correcto

El resultado de la ejecución fue `Tests run: 5, Failures: 0, Errors: 0, Skipped: 0` y `BUILD SUCCESS`. La salida se muestra en la Figura 20 de la sección 8.

5.3.11 Resumen de evidencias

Las evidencias de APF3 se organizan de la siguiente manera: base de datos y claves foráneas (Figura 6), modelo ER (Figura 5), CRUD de categorías (Figuras 7 a 9), inventario y CRUD de productos (Figuras 10 a 12), validaciones (Figuras 16 y 17), reportes DTO (Figura 13), selects en cascada (Figuras 14 y 15), reserva y stock (Figuras 18 y 19), pruebas (Figura 20) y persistencia después de reiniciar (Figura 21).

5.4 Entrega 4 – Proyecto Final: Spring Security

Esta sección corresponde a la entrega final y no forma parte del alcance del Tercer Avance. La aplicación actual utiliza un login con `HttpSession` que guarda el identificador del usuario; no implementa Spring Security, autorización por roles en el servidor ni tokens JWT.

Para la entrega final se deberá incorporar Spring Security, proteger las rutas `/administracion` y `/admin/**`, restringir los endpoints `/api/**`, cifrar las contraseñas con BCrypt y evaluar la incorporación de JWT para una API separada. Estas

funcionalidades se documentan como trabajo futuro y no se presentan como implementadas.

6. ARQUITECTURA DEL SISTEMA

La aplicación conserva el patrón MVC y se organiza en capas. El navegador solicita una URL; el controlador recibe la petición, invoca al servicio y agrega los datos al `Model`; el servicio aplica las reglas de negocio y las transacciones; el repositorio accede a MySQL mediante JPA/Hibernate; y Thymeleaf renderiza la vista HTML.

```
Navegador (Bootstrap + Thymeleaf)
              ↓ HTTP
Controller (MVC / REST)
              ↓
Service (@Service, @Transactional)
              ↓
Repository (JpaRepository)
              ↓
JPA / Hibernate
              ↓
MySQL (HikariCP)
```

Para los selects en cascada, el navegador ejecuta `fetch` contra `/api/productos` o `/api/reportes/reservas`; el controlador REST devuelve JSON y el JavaScript actualiza el select correspondiente sin recargar la página. Esta ruta convive con las vistas Thymeleaf y mantiene separadas las responsabilidades de presentación y de datos.

```
fetch('/api/productos?categoriaId=2')
        ↓
ProductoApiController → ProductoService → ProductoRepository
        ↓
MySQL → JSON → actualización del select
```

Las decisiones arquitectónicas principales son: usar formularios DTO para no exponer entidades JPA; usar DTO de lectura para los reportes; mantener el carrito como un objeto de sesión con snapshots de producto; y ubicar las transacciones de stock en los servicios, no en los controladores. El carrito es independiente por sesión HTTP y no se sincroniza entre dispositivos.

7. TECNOLOGÍAS UTILIZADAS

La selección de tecnologías busca mantener una curva de aprendizaje coherente con el curso y, al mismo tiempo, cubrir las necesidades de una aplicación web con persistencia relacional, validación, pruebas y generación de documentos.

Tabla 9. Tecnologías utilizadas y su propósito.

Tecnología	Uso dentro del proyecto
HTML5	Estructura de las vistas y formularios.

Tecnología	Uso dentro del proyecto
CSS3	Estilos personalizados y ajustes visuales.
JavaScript	Interacciones, modales y selects en cascada.
Bootstrap 5.3	Grillas, componentes, navegación y diseño responsivo.
Java 21	Lenguaje del backend y de las entidades.
Spring Boot 4.1.1	Configuración automática, arranque y servidor integrado.
Spring Web MVC	Controladores, rutas y endpoints REST.
Thymeleaf	Motor de plantillas del lado del servidor.
Spring Data JPA	Repositorios y consultas JPQL.
Hibernate	Implementación de JPA y mapeo objeto-relacional.
MySQL 8+ (26.7 en el entorno)	Base de datos relacional `korvia`.
HikariCP	Pool de conexiones de Spring Boot.
Jakarta Bean Validation	Validación de formularios en el servidor.
H2	Base de datos en memoria para las pruebas.
OpenPDF 1.3.30	Generación de boletas y cotizaciones en PDF.
Maven Wrapper	Compilación, pruebas y ejecución del proyecto.
Git / GitHub	Control de versiones y publicación del código.
Spring Security / JWT	Trabajo futuro; no implementado en APF3.

La aplicación no usa iText: la generación de PDF se realiza con OpenPDF. Tampoco se debe confundir `script.js` con el JavaScript activo, ya que la interacción actual está incorporada en las plantillas Thymeleaf.

8. PRUEBAS DEL SISTEMA

Las pruebas se dividen en automatizadas y manuales. Las automatizadas se ejecutan con H2 y Maven Wrapper; las manuales se realizaron sobre la aplicación en `http://localhost:8080` con MySQL y sesión de administrador.

8.1 Pruebas automatizadas

El comando `.\mvnw.cmd clean test` compila el proyecto y ejecuta cinco pruebas. El resultado obtenido fue `Tests run: 5, Failures: 0, Errors: 0, Skipped: 0` y `BUILD SUCCESS`. La salida completa se muestra en la Figura 20.

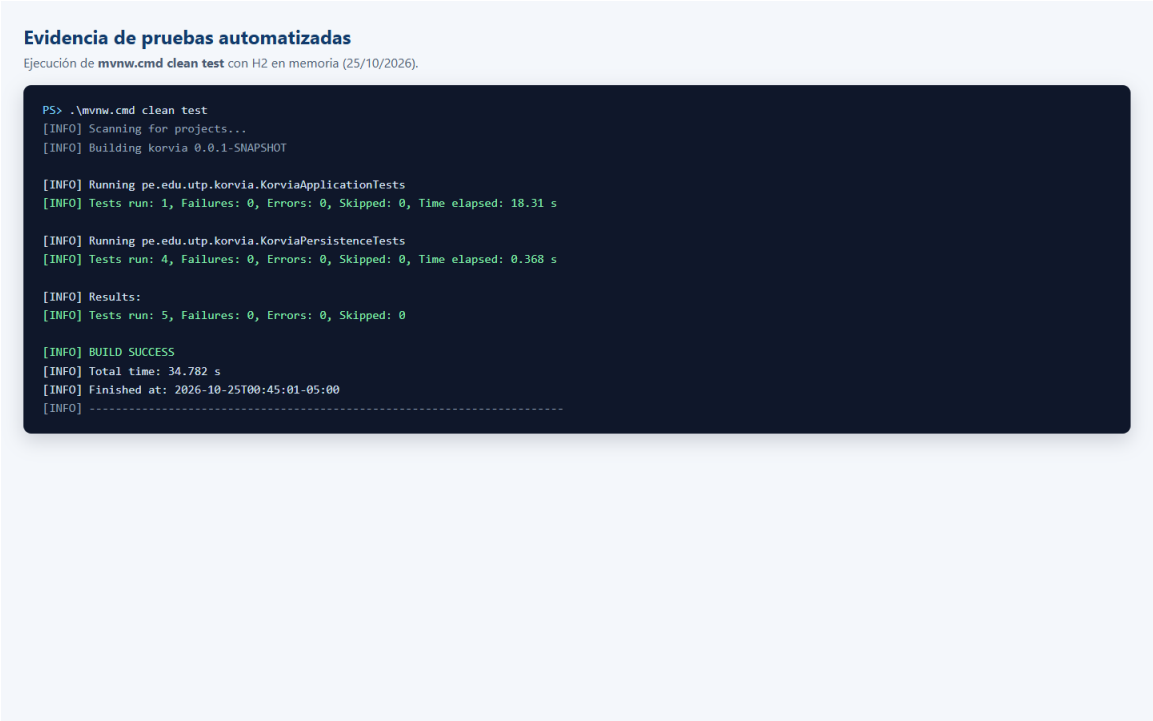


Figura 20. Salida de las pruebas automatizadas con Maven y H2. Fuente: ejecución local.

8.2 Pruebas manuales

Las pruebas manuales verificaron los flujos principales de la interfaz y la persistencia. La Tabla 10 resume los casos ejecutados y su resultado.

Tabla 10. Pruebas manuales y resultados.

N.°	Caso de prueba	Resultado esperado	Resultado obtenido
1	Inicio de sesión con admin@korvia.pe	Acceso al panel de administración.	Cumple
2	Listado de categorías	Mostrar categorías, productos, stock y disponible.	Cumple; Figura 7
3	Alta de categoría válida	Guardar y regresar al listado.	Cumple
4	Validación de categoría vacía	Mostrar mensajes sin guardar.	Cumple; Figura 16
5	Edición de categoría	Cargar datos y actualizar.	Cumple; Figura 9
6	Validación de producto vacío	Mostrar errores de nombre, precio, stock e imagen.	Cumple; Figura 17
7	Reserva de producto	Incrementar reservado y reducir disponible.	Cumple; Figuras 18 y 19
8	Selects en cascada en /reservas	Cargar productos de la categoría elegida.	Cumple; Figura 14
9	Reporte DTO y filtros en cascada	Mostrar tablas y filtrar reservas.	Cumple; Figuras 13 y 15

N.°	Caso de prueba	Resultado esperado	Resultado obtenido
10	Reinicio de la aplicación	Conservar la reserva y los datos en MySQL.	Cumple; Figura 21

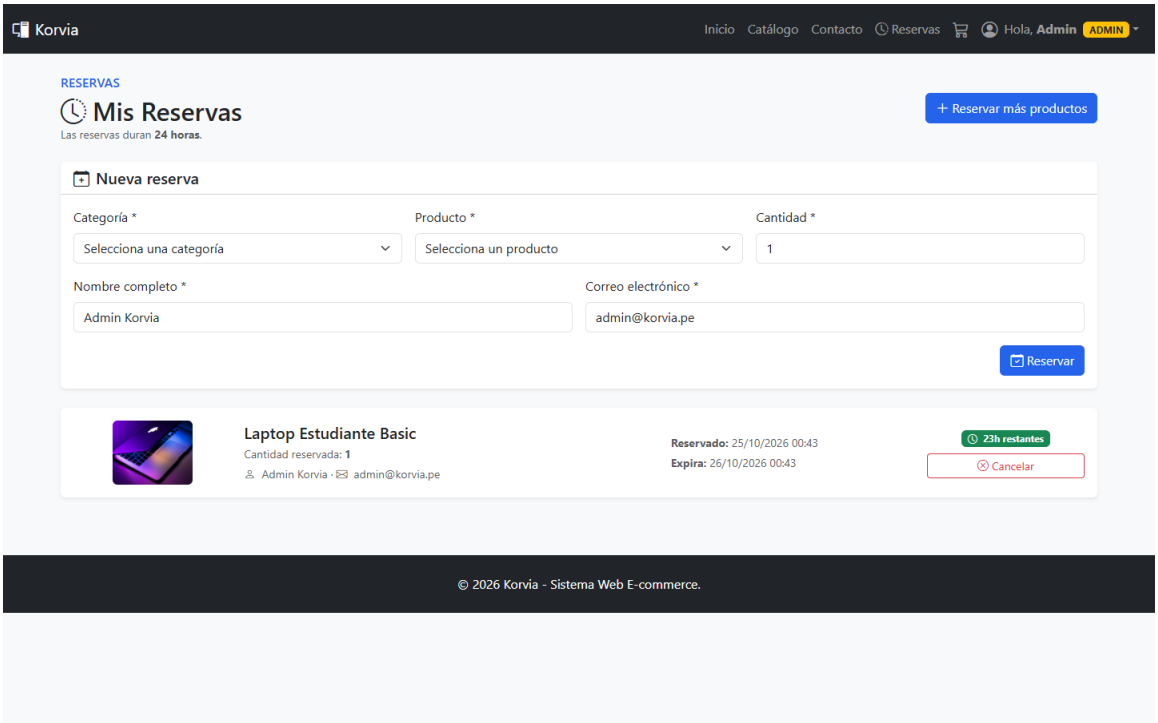


Figura 21. Reserva conservada después de reiniciar la aplicación. Fuente: captura del sistema.

8.3 Resultados

La aplicación arranca con MySQL, crea o actualiza las tablas y conserva los datos después de reiniciar. Los CRUD de categorías y productos funcionan con validación; las consultas DTO devuelven datos combinados de varias tablas; los selects en cascada responden a las selecciones; y el stock se mantiene consistente en las reservas y cancelaciones. Las cinco pruebas automatizadas con H2 finalizaron sin fallos.

9. CONCLUSIONES

La migración de la aplicación Korvia a MySQL y Spring Data JPA permitió reemplazar el almacenamiento en memoria por una base relacional con integridad referencial. Las cinco tablas y sus claves foráneas sostienen el catálogo, los usuarios, las categorías, los productos y las reservas, y los datos se conservan después de reiniciar la aplicación.

Las consultas multitabla con DTO y los selects en cascada mejoran la separación de capas y la experiencia de usuario. Los reportes no exponen entidades JPA y los formularios solo reciben los datos necesarios; además, los filtros dependientes evitan combinaciones inválidas entre categoría, producto y reserva.

Las validaciones de Jakarta Bean Validation, las reglas de negocio en los servicios y las transacciones de stock evitan datos inconsistentes, códigos duplicados y reservas que superen la disponibilidad. Las pruebas con H2 y las pruebas funcionales sobre MySQL proporcionan evidencia verificable del comportamiento del sistema.

Como trabajo futuro queda persistir boletas y cotizaciones, incorporar Spring Security con BCrypt y autorización por roles, agregar paginación y auditoría de stock, y desplegar la aplicación en un entorno cloud con respaldos y monitoreo.

10. RECOMENDACIONES

- Persistir boletas y cotizaciones en MySQL para que el historial comercial no se pierda al reiniciar la aplicación.
- Implementar Spring Security con BCrypt, restricción de rutas y autorización por roles; evaluar JWT solo si se separa una API para clientes externos.
- Agregar paginación, búsqueda combinada y ordenamiento en los listados de productos y reservas.
- Implementar un historial de movimientos de stock para auditar reservas, cancelaciones, ventas y ajustes manuales.
- Incorporar Flyway o Liquibase para versionar el esquema de la base de datos y reducir la dependencia de `ddl-auto=update`.
- Programar respaldos periódicos de MySQL y monitorear el pool HikariCP en un entorno de producción.
- Desplegar la aplicación en un servicio cloud con HTTPS, variables de entorno y separación de configuraciones por ambiente.

11. ANEXOS

11.1 Modelo entidad-relación

El modelo completo se presenta en la Figura 5 y en el archivo `docs/avance3/modelo-er.md`. Las relaciones principales son `roles` 1:N `usuarios`, `categorias` 1:N `productos`, `productos` 1:N `reservas` y `usuarios` 1:N `reservas` (opcional).

11.2 Script de la base de datos

El script completo se encuentra en `database/korvia.sql`. Es idempotente para roles, categorías, usuarios y productos, y permite crear la base `korvia` desde cero con UTF-8. A continuación se incluye el DDL principal de las cinco tablas.

```
CREATE DATABASE IF NOT EXISTS korvia
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE korvia;
SET NAMES utf8mb4;

CREATE TABLE IF NOT EXISTS roles (
    id BIGINT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uk_roles_nombre (nombre)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS categorias (
    id BIGINT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    codigo VARCHAR(50) NOT NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uk_categorias_nombre (nombre),
    UNIQUE KEY uk_categorias_codigo (codigo)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS usuarios (
    id BIGINT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL,
    password VARCHAR(255) NOT NULL,
    rol_id BIGINT NOT NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uk_usuarios_email (email),
    CONSTRAINT fk_usuarios_rol FOREIGN KEY (rol_id) REFERENCES roles (id)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS productos (
    id BIGINT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(200) NOT NULL,
    descripcion VARCHAR(500) NOT NULL,
    descripcion_larga TEXT,
    precio DOUBLE NOT NULL,
    stock INT NOT NULL,
    reservado INT NOT NULL DEFAULT 0,
    imagen VARCHAR(255),
    categoria_id BIGINT NOT NULL,
    cpu VARCHAR(100),
    ram VARCHAR(100),
    almacenamiento VARCHAR(100),
    gpu VARCHAR(100),
    PRIMARY KEY (id),
    CONSTRAINT fk_productos_categoria FOREIGN KEY (categoria_id) REFERENCES categorias (id)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS reservas (
    id BIGINT NOT NULL AUTO_INCREMENT,
    producto_id BIGINT NOT NULL,
    usuario_id BIGINT NULL,
    cliente_nombre VARCHAR(150) NOT NULL,
```

```
    cliente_email VARCHAR(150) NOT NULL,
    cantidad INT NOT NULL,
    fecha_reserva DATETIME(6) NOT NULL,
    fecha_expiracion DATETIME(6) NOT NULL,
    estado VARCHAR(20) NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_reservas_producto FOREIGN KEY (producto_id) REFERENCES productos (id),
    CONSTRAINT fk_reservas_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios (id) ON DE
LETE SET NULL
) ENGINE=InnoDB;
```

11.3 Repositorio y ejecución

Repositorio: https://github.com/Davi-to/Proyecto_MarcosWeb

Para ejecutar la aplicación se requiere MySQL, Java 17 o superior y el Maven Wrapper incluido. La contraseña se define en la variable de entorno `DB_PASSWORD`.

```
$env:DB_PASSWORD='tu_password_local'
.\mvnw.cmd spring-boot:run

# Pruebas automatizadas con H2
.\mvnw.cmd clean test
```

11.4 Credenciales de prueba

- Administrador: `admin@korvia.pe` / `admin123`.
- Cliente: `juan@email.com` / `123456`.

Las contraseñas se almacenan en texto plano en esta versión académica. No deben reutilizarse en un entorno real; la entrega final debe incorporar BCrypt y Spring Security.

11.5 Evidencias complementarias

Las capturas utilizadas en el informe se encuentran en `docs/avance3/capturas/`. El repositorio incluye el código fuente, el script `database/korvia.sql`, el README con instrucciones y la documentación del avance.